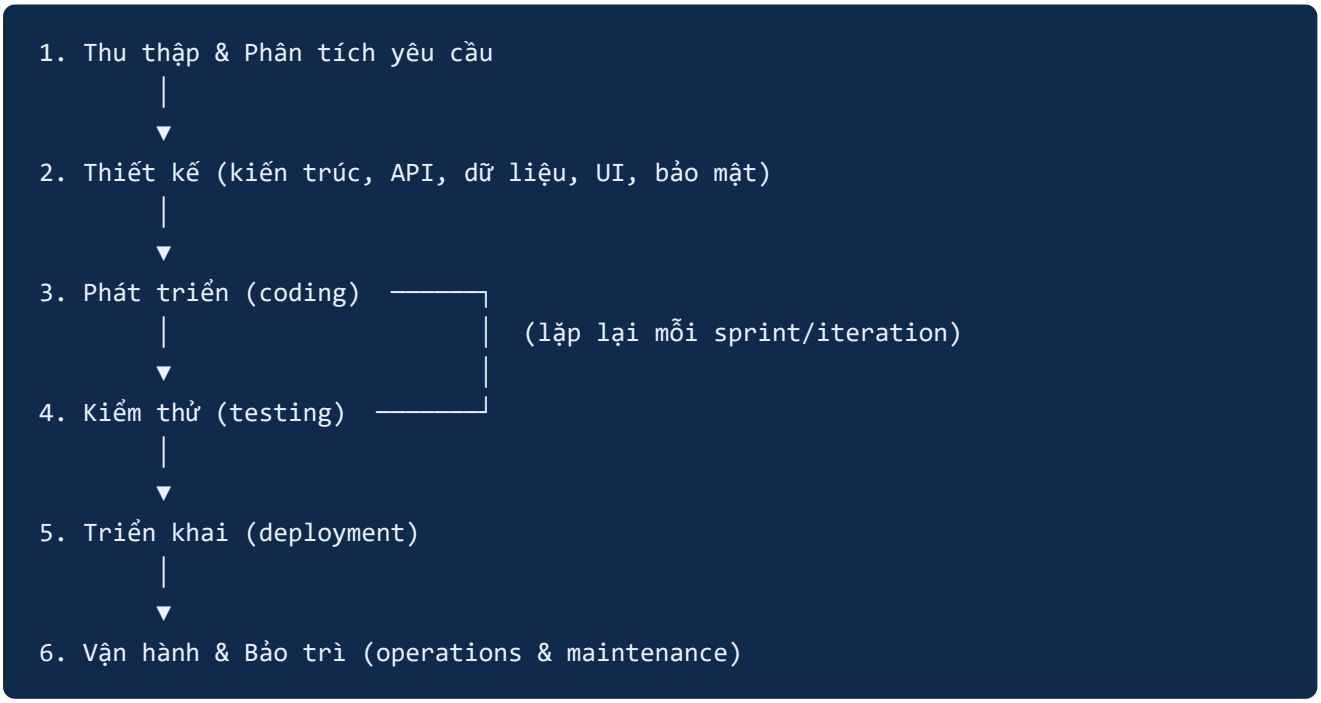
Hướng dẫn đặt prompt để sản xuất phần mềm đúng quy trình SDLC

Dựa trên bộ 45 agent & skill plugin `developer` — dev-quality-toolkit · Cập nhật 2026-10-08

0. Nguyên tắc chung khi đặt prompt

1. **Một prompt — một agent, một việc.** Nếu công việc trải nhiều giai đoạn (vd. "làm rõ yêu cầu rồi thiết kế luôn"), tách thành nhiều lượt hỏi tuần tự, đừng gộp vào một câu.
2. **Nêu rõ trạng thái hiện tại** — "chưa có code", "kiến trúc đã chốt chưa", "migration đã viết ra chưa". Đây là trục Claude Code dùng để chọn đúng agent giữa các cặp dễ nhầm (vd. `threat-modeler` trước code vs `security-auditor` sau code đã viết).
3. **Nêu rõ file/module cụ thể** thay vì mô tả chung chung — "review giúp file `order.py`" tốt hơn "review code giúp".
4. **Phần lớn agent ở giai đoạn Thiết kế/Kiểm thử/Vận hành chỉ phân tích & báo cáo**, không tự sửa file — nếu cần hành động, nối thêm yêu cầu dùng agent có quyền Write/Edit tương ứng.
5. **Không chắc nên dùng agent nào?** Gọi `/developer:agent-advisor` kèm mô tả việc — nó đọc trực tiếp `plugin.json` để tư vấn đúng theo hiện trạng repo.

1. Tổng quan pipeline 6 giai đoạn



Phần mềm hiện đại hiếm khi đi qua 6 giai đoạn này một lần rồi kết thúc — đây là một vòng lặp: mỗi tính năng mới, mỗi bản sửa lỗi đều đi qua lại từ bước 2 đến bước 6.

2. Giai đoạn 1 — Thu thập & Phân tích yêu cầu

Tình huống	Agent
Yêu cầu nghiệp vụ còn mơ hồ, cần user story + acceptance criteria	`requirements-analyst`
Yêu cầu đã rõ, cần văn bản hóa thành SRS chính thức	`srs-writer`

> *"Sếp tôi muốn có tính năng 'thông báo cho khách hàng khi đơn hàng có vấn đề' nhưng nói còn rất mơ hồ, giúp tôi làm rõ thành user story."*

> *"requirements-analyst đã làm rõ xong yêu cầu export đơn hàng ra CSV, giúp tôi viết thành tài liệu SRS chính thức."*

3. Giai đoạn 2 — Thiết kế

Tình huống	Agent
Kiến trúc **chưa chốt**, cần so sánh phương án + trade-off	`system-architect`
Kiến trúc **đã chốt**, cần tài liệu hóa High-Level Design	`high-level-designer`
Đã có HLD, cần chi tiết hóa class/sequence/pseudocode	`low-level-designer`
Thiết kế endpoint REST/GraphQL mới	`api-designer`
Thiết kế ERD mới (chưa có migration nào)	`data-modeler`
Review migration/schema **đã viết ra**	`db-schema-reviewer`
Đặc tả wireframe/luồng màn hình trước khi code UI	`ui-ux-designer`
Threat modeling (STRIDE) trước khi có code	`threat-modeler`
Ghi lại quyết định kiến trúc đã chốt	`adr-writer`
Đánh giá PoC/spike đã thực hiện, viết PoC report	`poc-evaluator`
Ước lượng effort (person-day) cho yêu cầu đã rõ	`project-estimator`
Dựng Work Breakdown Structure từ effort đã có	`wbs-writer`
Soạn Test Plan trước khi viết test case thật	`test-plan-writer`

> *"Lên kế hoạch triển khai tính năng thông báo real-time, cân nhắc WebSocket hay polling."* → *system-architect*

> *"HLD đã chốt dùng message queue cho luồng xử lý đơn hàng, giúp thiết kế chi tiết class/sequence cho module OrderProcessor."* → *low-level-designer*

> *"Trước khi code, giúp tôi threat model luồng thanh toán mới: client gọi API tạo order, service order gọi service payment qua message queue."* → *threat-modeler*

4. Giai đoạn 3 — Phát triển (Coding)

Tình huống	Agent
Review commit/diff Android (Kotlin/Java/Compose)	`android-commit-reviewer`
Review commit/diff Qt/C++	`qt-cpp-commit-reviewer`
Review code Python (PEP8 + bảo mật OWASP)	`python-pep8-security-reviewer`
Đánh giá kiến trúc code hiện có (coupling, SOLID)	`architecture-reviewer`
Refactor an toàn, không đổi hành vi	`refactor-assistant`
Viết unit test cho code hiện có	`test-writer`
Rà soát package bên thứ 3 (CVE, license, lỗi thời)	`dependency-auditor`
Nâng cấp version package/framework cụ thể	`dependency-upgrader`
Quét & ưu tiên hóa nợ kỹ thuật toàn repo	`tech-debt-tracker`
Sinh/cập nhật tài liệu API từ code (OpenAPI, docstring)	`api-doc-generator`
Viết commit message theo convention rồi commit	skill `/developer:commit`
Viết mô tả Pull Request từ diff	`pr-description-writer`

> *"Review giúp commit vừa rồi sửa màn hình login bằng Kotlin coroutine."* → *android-commit-reviewer*

> *"Refactor giúp function xử lý đơn hàng trong order.py đang quá dài và lồng nhau nhiều if."* → *refactor-assistant*

5. Giai đoạn 4 — Kiểm thử

Tình huống	Agent
Viết/chạy integration hoặc E2E test	`e2e-test-runner`
Phân tích coverage, tìm vùng thiếu test cần ưu tiên	`test-coverage-analyzer`
Test lúc pass lúc fail (flaky), cần tìm nguyên nhân gốc	`flaky-test-detector`

Audit lỗ hổng bảo mật trong code đã viết	`security-auditor`
Profile hiệu năng: bottleneck, N+1 query, memory leak	`performance-profiler`

> *"Viết integration test cho API tạo đơn hàng, cần test cả trường hợp kho hết hàng."* → *e2e-test-runner*

6. Giai đoạn 5 — Triển khai

Tình huống	Agent
Tối ưu Dockerfile (size, cache, bảo mật)	`docker-optimizer`
Review Terraform/K8s/Helm trước khi apply	`iac-reviewer`
Chẩn đoán & sửa pipeline CI đang fail	`ci-fixer`
Kiểm tra cấu hình pre-flight trước khi deploy	`deployment-validator`
Checklist go/no-go + kế hoạch rollback cho 1 release	`release-manager`

> *"Kiểm tra giúp cấu hình trước khi deploy service payment lên staging."* → *deployment-validator*

7. Giai đoạn 6 — Vận hành & Bảo trì

Tình huống	Agent
Sự cố production **đang diễn ra** ngay bây giờ	`incident-commander`
Cần tìm pattern bất thường trên tập log diện rộng	`log-analyzer`
Đã có stack trace/bug report cụ thể, cần truy nguyên nhân gốc	`root-cause-analyzer`
Tổng hợp CHANGELOG giữa 2 mốc release	`changelog-writer`
Giải thích codebase cho thành viên mới	`onboarding-guide`
Review chất lượng tài liệu Markdown/README	`docs-reviewer`
Viết mới/cập nhật README, hướng dẫn cài đặt	`doc-writer`

> *"Production đang down, error rate tăng vọt 5 phút trước, giúp tôi xác định mức độ nghiêm trọng và việc cần làm ngay."* → *incident-commander*

8. Hoạt động xuyên suốt toàn bộ quy trình

Việc cần làm	Agent / Skill
Điều phối reviewer theo stack, gộp checklist PASS/WARN/FAIL trước merge	skill `/developer:pre-merge`
Đối chiếu dự án/tính năng với checklist quality gate (ADR, test, security audit, changelog...)	`qms-compliance-checker`
Nối từng ID yêu cầu với thiết kế/code/PR/test case (RTM)	`rtm-manager`
Tư vấn nên dùng agent nào cho một việc cụ thể	skill `/developer:agent-advisor`

9. Tóm tắt nhanh — chọn công cụ theo tình huống

Tôi đang...	Nên dùng
Lên kế hoạch cho một tính năng lớn chưa có	`system-architect`
Chuẩn bị merge một PR	agent review theo stack + `pr-description-writer`
Code chạy chậm hoặc nghi ngờ leak	`performance-profiler`
Nghi ngờ lỗ hổng bảo mật trong code đã viết	`security-auditor`
Cần soát rủi ro bảo mật TRƯỚC khi code	`threat-modeler`
Cần số effort cụ thể cho yêu cầu đã rõ	`project-estimator`
Chuẩn bị release	`changelog-writer` → `release-manager` → `deployment-validator`
Production đang down NGAY BÂY GIỜ	`incident-commander`
Production ổn định, cần tìm nguyên nhân gốc	`log-analyzer` → `root-cause-analyzer`
Codebase nhiều nợ kỹ thuật, không biết bắt đầu từ đâu	`tech-debt-tracker`
Thành viên mới cần hiểu dự án	`onboarding-guide`
Yêu cầu nghiệp vụ còn mơ hồ	`requirements-analyst`
Kiểm tra tính năng đã đủ hồ sơ chất lượng chưa	`qms-compliance-checker`

Nguồn: plugin `developer` v0.4.0 — dev-quality-toolkit · Tài liệu gốc: `SDLC/QUY_TRINH_SAN_XUAT_PHAN_MEM.md`, `SDLC/DANH_SACH_AI_AGENT_VA_VI_DU_PROMPT.md`